

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №2
дисциплины
«Искусственный интеллект и машинное обучение»

Выполнил:
Хубиев Роберт Эльбрусович
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверила:
Доцент департамента цифровых,
робототехнических систем и
электроники института
перспективной инженерии Воронкин
Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Цель работы: исследовать базовые возможности библиотеки NumPy языка программирования Python.

Ссылка на репозиторий: <https://github.com/truebobsuncle/IE2>

Ход работы:

1. Создан общедоступный репозиторий, в котором использована лицензия MIT и язык программирование Python.

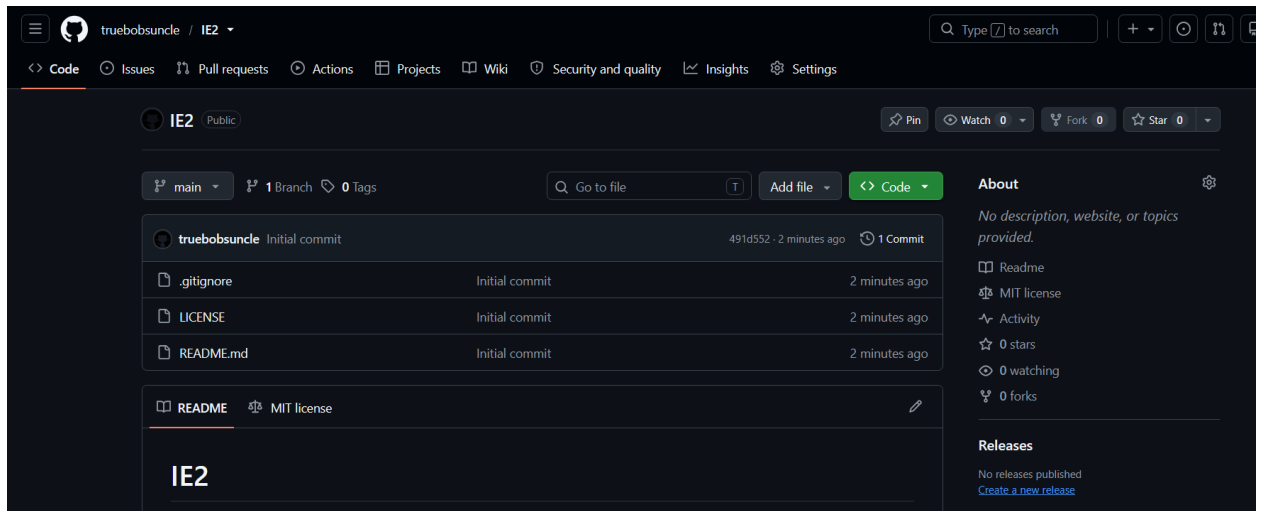


Рисунок 1. Созданный репозиторий

2. Клонирование репозитория.

```
Роберт@KOMP MINGW64 ~ (main)
$ git clone https://github.com/truebobsuncle/IE2.git
Cloning into 'IE2'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.
```

Рисунок 2. Клонирование репозитория

3. .gitignore был дополнен необходимыми правилами.

```

# PyBuilder
.pybuilder/
target/

# Jupyter Notebook
.ipynb_checkpoints/
*.ipynb_checkpoints*
*/.ipynb_checkpoints/*
.ipynb_checkpoints/*
*.ipynb.json
*/.ipynb_checkpoints/
~*.ipynb
*.ipynb.py
*.ipynb_checkpoints/
*.pkl
*.pickle

# JupyterLab
.jupyter/
jupyter_config.json
jupyter_notebook_config.json
jupyter_notebook_config.py
jupyterlab-workspace.json

# PyCharm
.idea/
*.iml
*.iws
.idea_modules/
.idea/
out/

```

Рисунок 3. Дополнили. gitignore

4. Были проработаны примеры лабораторной работы.

```

[1]: import numpy as np
      a = np.array([1, 2, 3])
      b = np.array([4, 5, 6])
      result = a + b
      print(result)

[5 7 9]

```

Рисунок 4. Пример 1

```
[10]: import numpy as np
      arr = np.array([1, 2, 3, 4])
```

```
[11]: import numpy as np
      arr1 = np.array([1, 2, 3])
      arr2 = np.array([4, 5, 6])
      result = arr1 + arr2
      print(result)
```

[5 7 9]

```
[12]: import numpy as np
      matrix1 = np.array([[1, 2], [3, 4]])
      matrix2 = np.array([[5, 6], [7, 8]])
      result = np.dot(matrix1, matrix2)
      print(result)
```

[[19 22]
 [43 50]]

```
[13]: data = np.array([1, 2, 3, 4, 5])
      mean = np.mean(data)
      std_dev = np.std(data)
```

Рисунок 5. Пример 2

```
[2]: import numpy as np
      arr = np.array([1, 2, 3, 4])
      result = arr * 2
      print(result)
```

[2 4 6 8]

Рисунок 6. Пример 3

```
[10]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(m)

[[1 2 3 4]
 [5 6 7 8]
 [9 1 5 7]]
```

```
[12]: import numpy as np
n = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(n[1, 0])

5
```

```
[11]: import numpy as np
n = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(n[1, :])

[[5 6 7 8]]
```

```
[13]: import numpy as np
n = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(n[:, 2])

[[3]
 [7]
 [5]]
```

```
[14]: import numpy as np
n = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(n[1, 2:])

[[7 8]]
```

```
[15]: import numpy as np
n = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(n[0:2, 1])

[[2]
 [6]]
```

```
[16]: import numpy as np
n = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(n[0:2, 1:3])

[[2 3]
 [6 7]]
```

```
[17]: import numpy as np
n = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
cols = [0, 1, 3]
print(n[:, cols])

[[1 2 4]
 [5 6 8]
 [9 1 7]]
```



Рисунок 7. Пример 4

```
[14]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(m)

[[1 2 3 4]
 [5 6 7 8]
 [9 1 5 7]]
```

```
[18]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(type(m))
print(m)

<class 'numpy.matrix'>
[[1 2 3 4]
 [5 6 7 8]
 [9 1 5 7]]
```

```
[19]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
m = np.array(m)
print(type(m))

<class 'numpy.ndarray'>
```

```
[20]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(m.shape)

(3, 4)
```

```
[22]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(m.max())

9
```

```
[24]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(np.max(m))

9
```

```
[25]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(m.max())

9
```

```
[26]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(m.max(axis=1))

[[4]
 [8]
 [9]]
```

```
[27]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(m.max(axis=0))

[[9 6 7 8]]
```

```
[31]: import numpy as np
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print(m.mean())
print(m.mean(axis=1))
print(m.sum())
print(m.sum(axis=0))

4.833333333333333
[[2.5]
 [6.5]
 [5.5]]
58
[[15 9 15 19]]
```

Рисунок 8. Пример 5

```
[4]: import numpy as np
      nums = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
      letters = np.array(['a', 'b', 'c', 'd', 'a', 'c', 'b'])
      print(nums)
      print(letters)

[ 1  2  3  4  5  6  7  8  9 10]
['a' 'b' 'c' 'd' 'a' 'c' 'b']

[8]: import numpy as np
      a = True
      print(a)

True

[9]: b = 5 > 7
      print(b)

False

[10]: import numpy as np
       nums = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
       letters = np.array(['a', 'b', 'c', 'd', 'a', 'c', 'b'])
       less_than_5 = nums < 5
       print(less_than_5)

[ True  True  True  True False False False False False]

[11]: import numpy as np
       nums = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
       letters = np.array(['a', 'b', 'c', 'd', 'a', 'c', 'b'])
       pos_a = letters == 'a'
       print(pos_a)

[ True False False False  True False False]

[12]: import numpy as np
       nums = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
       letters = np.array(['a', 'b', 'c', 'd', 'a', 'c', 'b'])
       less_than_5 = nums < 5
       print(nums[less_than_5])

[1 2 3 4]

[16]: import numpy as np
       m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
       mod_m = np.logical_and(m>=3, m<=7)
       print(mod_m)
       print(m[mod_m])

[[False False  True  True]
 [ True  True  True False]
 [False False  True  True]]
[[3 4 5 6 7 5 7]]

[19]: import numpy as np
       nums = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
       print(nums[nums < 5])

[1 2 3 4]

[20]: import numpy as np
       nums = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
       nums[nums < 5] = 10
       print(nums)

[10 10 10 10  5  6  7  8  9 10]

[22]: import numpy as np
       m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
       m[m > 7] = 25
       print(m)

[[ 1  2  3  4]
 [ 5  6  7 25]
 [25  1  5  7]]
```

Рисунок 9. Пример 6

```
[2]: import numpy as np
      print(np.arange(10))

[0 1 2 3 4 5 6 7 8 9]

[3]: import numpy as np
      print(np.arange(5, 12))

[ 5  6  7  8  9 10 11]

[4]: import numpy as np
      print(np.arange(1, 5, 0.5))

[1.  1.5 2.  2.5 3.  3.5 4.  4.5]
```

Рисунок 10. Пример 7

```
[2]: import numpy as np
      a = [[1, 2], [3, 4]]
      print(np.matrix(a))

[[1 2]
 [3 4]]

[5]: import numpy as np
      b = np.array([[5, 6], [7, 8]])
      print(np.matrix(b))

[[5 6]
 [7 8]]

[7]: import numpy as np
      m = np.matrix('[1, 2; 3, 4]')
      print(m)

[[1 2]
 [3 4]]
```

Рисунок 11. Пример 8

```
[2]: import numpy as np
      print(np.zeros((3, 4)))

[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]

[3]: import numpy as np
      print(np.eye(3))

[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```


Рисунок 12. Пример 9

```
[2]: import numpy as np
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(A)

[[1 2 3]
 [4 5 6]
 [7 8 9]]

[3]: import numpy as np
print(np.ravel(A))

[1 2 3 4 5 6 7 8 9]

[4]: import numpy as np
print(np.ravel(A, order='C'))

[1 2 3 4 5 6 7 8 9]

[5]: import numpy as np
print(np.ravel(A, order='F'))

[1 4 7 2 5 8 3 6 9]
```

Рисунок 13. Пример 10

```
[1]: import numpy as np
a = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
print(np.where(a % 2 == 0, a * 10, a / 10))

[ 0.   0.1 20.   0.3 40.   0.5 60.   0.7 80.   0.9]

[3]: import numpy as np
a = np.random.rand(10)
print(a)
print(np.where(a > 0.5, True, False))
print(np.where(a > 0.5, 1, -1))

[0.9878375  0.53983339 0.35622593 0.22043178 0.91254446 0.54900335
 0.49454998 0.07035035 0.55880495 0.95666549]
[ True  True False False  True  True False False  True  True]
[ 1  1 -1 -1  1  1 -1 -1  1  1]
```

Рисунок 14. Пример 11

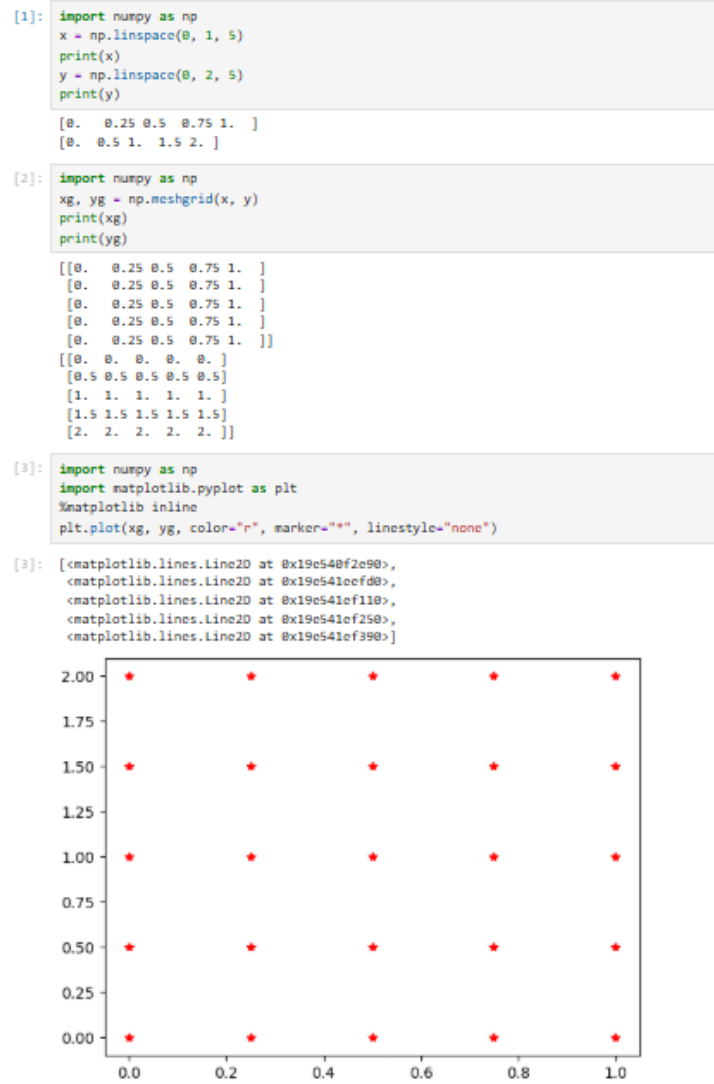


Рисунок 15. Пример 12

```
[1]: import numpy as np
print(np.random.permutation(7))
a = ['a', 'b', 'c', 'd', 'e']
print(np.random.permutation(a))

[6 4 1 3 0 5 2]
['d' 'a' 'b' 'c' 'e']

[2]: import numpy as np
arr = np.linspace(0, 10, 5)
print(arr)

[ 0.  2.5  5.  7.5 10.]

[3]: import numpy as np
arr_mix = np.random.permutation(arr)
print(arr_mix)

[ 2.5  7.5  0. 10.  5.]

[4]: import numpy as np
index_mix = np.random.permutation(len(arr_mix))
print(index_mix)
print(arr[index_mix])

[1 0 4 3 2]
[ 2.5  0. 10.  7.5  5.]
```

Рисунок 16. Пример 13

```

[1]: import numpy as np
    v_hor_np = np.array([1, 2])
    print(v_hor_np)

[1 2]

[2]: v_hor_zeros_v1 = np.zeros((5, ))
    print(v_hor_zeros_v1)

[0. 0. 0. 0. 0.]

[3]: v_hor_one_v1 = np.ones((5, ))
    print(v_hor_one_v1)
    v_hor_one_v2 = np.ones((1, 5))
    print(v_hor_one_v2)

[1. 1. 1. 1. 1.]
[[1. 1. 1. 1. 1.]]

[4]: v_vert_np = np.array([[1], [2]])
    print(v_vert_np)

[[1]
 [2]]

[5]: v_vert_zeros = np.zeros((5, 1))
    print(v_vert_zeros)

[[0.]
 [0.]
 [0.]
 [0.]
 [0.]]

[6]: v_vert_ones = np.ones((5, 1))
    print(v_vert_ones)

[[1.]
 [1.]
 [1.]
 [1.]
 [1.]]

```

Рисунок 17. Пример 14

```
[2]: import numpy as np
      m_sqr_arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
      print(m_sqr_arr)

      [[1 2 3]
       [4 5 6]
       [7 8 9]]

[9]: m_sqr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
      m_sqr_arr = np.array(m_sqr)
      print(m_sqr_arr)

      [[1 2 3]
       [4 5 6]
       [7 8 9]]

[7]: m_sqr_mx = np.matrix([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
      print(m_sqr_mx)

      [[1 2 3]
       [4 5 6]
       [7 8 9]]

[10]: m_sqr_mx = np.matrix('1 2 3; 4 5 6; 7 8 9')
      print(m_sqr_mx)

      [[1 2 3]
       [4 5 6]
       [7 8 9]]
```

Рисунок 18. Пример 15

```
[1]: import numpy as np
      m_diag = [[1, 0, 0], [0, 5, 0], [0, 0, 9]]
      m_diag_np = np.matrix(m_diag)
      print(m_diag_np)

      [[1 0 0]
       [0 5 0]
       [0 0 9]]

[3]: m_sqr_mx = np.matrix('1 2 3; 4 5 6; 7 8 9')

[4]: diag = np.diag(m_sqr_mx)
      print(diag)

      [1 5 9]

[5]: m_diag_np = np.diag(np.diag(m_sqr_mx))
      print(m_diag_np)

      [[1 0 0]
       [0 5 0]
       [0 0 9]]
```

Рисунок 19. Пример 16

```
[2]: import numpy as np
m_e = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]
m_e_np = np.matrix(m_e)
print(m_e_np)
```

```
[[1 0 0]
 [0 1 0]
 [0 0 1]]
```

```
[4]: m_eye = np.eye(3)
print(m_eye)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
[5]: m_idnt = np.identity(3)
print(m_idnt)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Рисунок 20. Пример 17

```
[1]: import numpy as np
m_zeros = np.zeros((3, 3))
print(m_zeros)
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

Рисунок 21. Пример 18

```
[1]: import numpy as np
m_mx = np.matrix('1 2 3; 4 5 6')
print(m_mx)
```

```
[[1 2 3]
 [4 5 6]]
```

```
[2]: m_var = np.zeros((2, 5))
print(m_var)
```

```
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]]
```

Рисунок 22. Пример 19

```
[1]: import numpy as np
A = np.matrix('1 2 3; 4 5 6')
print(A)

[[1 2 3]
 [4 5 6]]
```

```
[2]: A_t = A.transpose()
print(A_t)

[[1 4]
 [2 5]
 [3 6]]
```

```
[4]: print(A.T)

[[1 4]
 [2 5]
 [3 6]]
```

```
[5]: A = np.matrix('1 2 3; 4 5 6')
print(A)
R = (A.T).T
print(R)

[[1 2 3]
 [4 5 6]]
[[1 2 3]
 [4 5 6]]
```

```
[7]: A = np.matrix('1 2 3; 4 5 6')
B = np.matrix('7 8 9; 0 7 5')
L = (A + B).T
R2 = A.T + B.T
print(L)
print(R)

[[ 8  4]
 [10 12]
 [12 11]]
[[1 2 3]
 [4 5 6]]
```

```
[8]: A = np.matrix('1 2; 3 4')
B = np.matrix('5 6; 7 8')
L = (A.dot(B)).T
R = (B.T).dot(A.T)
print(L)
print(R)

[[19 43]
 [22 50]]
[[19 43]
 [22 50]]
```

```
[9]: A = np.matrix('1 2 3; 4 5 6')
k = 3
L = (k * A).T
R = k * (A.T)
print(L)
print(R)

[[ 3 12]
 [ 6 15]
 [ 9 18]]
[[ 3 12]
 [ 6 15]
 [ 9 18]]
```

```
[10]: A = np.matrix('1 2; 3 4')
A_det = np.linalg.det(A)
A_T_det = np.linalg.det(A.T)
print(f'{A_det:.9f}')
print(f'{A_T_det:.9f}')

-2.000000000
-2.000000000
```

Рисунок 23. Пример 20

```
[1]: import numpy as np
A = np.matrix('1 2 3; 4 5 6')
C = 3 * A
print(C)

[[ 3  6  9]
 [12 15 18]]
```

```
[2]: A = np.matrix('1 2; 3 4')
L = 1 * A
R = A
print(L)
print(R)

[[1 2]
 [3 4]]
[[1 2]
 [3 4]]
```

```
[3]: A = np.matrix('1 2; 3 4')
Z = np.matrix('0 0; 0 0')
L = 0 * A
R = Z
print(L)
print(R)

[[0 0]
 [0 0]]
[[0 0]
 [0 0]]
```

```
[5]: A = np.matrix('1 2; 3 4')
p = 2
q = 3
L = (p + q) * A
R = p * A + q * A
print(L)
print(R)

[[ 5 10]
 [15 20]]
[[ 5 10]
 [15 20]]
```

```
[7]: A = np.matrix('1 2; 3 4')
p = 2
q = 3
L = (p * q) * A
R = p * (q * A)
print(L)
print(R)

[[ 6 12]
 [18 24]]
[[ 6 12]
 [18 24]]
```

```
[8]: A = np.matrix('1 2; 3 4')
B = np.matrix('5 6; 7 8')
k = 3
L = k * (A + B)
R = k * A + k * B
print(L)
print(R)

[[18 24]
 [30 36]]
[[18 24]
 [30 36]]
```

Рисунок 24. Пример 21 (1 часть)


```
[9]: A = np.matrix('1 2; 3 4')
      B = np.matrix('5 6; 7 8')
      C = A + B
      print(C)

[[ 6  8]
 [10 12]]
```

```
[10]: A = np.matrix('1 2; 3 4')
      B = np.matrix('5 6; 7 8')
      L = A + B
      R = B + A
      print(L)
      print(R)

[[ 6  8]
 [10 12]]
[[ 6  8]
 [10 12]]
```

```
[11]: A = np.matrix('1 2; 3 4')
      B = np.matrix('5 6; 7 8')
      C = np.matrix('1 7; 9 3')
      L = A + (B + C)
      R = (A + B) + C
      print(L)
      print(R)

[[ 7 15]
 [19 15]]
[[ 7 15]
 [19 15]]
```

```
[12]: A = np.matrix('1 2; 3 4')
      Z = np.matrix('0 0; 0 0')
      L = A + (-1)*A
      print(L)
      print(Z)

[[0 0]
 [0 0]]
[[0 0]
 [0 0]]
```

Рисунок 25. Пример 21 (2 часть)

```
[3]: import numpy as np
A = np.matrix('1 2 3; 4 5 6')
B = np.matrix('7 8; 9 1; 2 3')
C = A.dot(B)
print(C)

[[31 19]
 [85 55]]
```

```
[4]: A = np.matrix('1 2; 3 4')
B = np.matrix('5 6; 7 8')
C = np.matrix('2 4; 7 8')
L = A.dot(B.dot(C))
R = (A.dot(B)).dot(C)
print(L)
print(R)

[[192 252]
 [436 572]]
[[192 252]
 [436 572]]
```

```
[5]: A = np.matrix('1 2; 3 4')
B = np.matrix('5 6; 7 8')
C = np.matrix('2 4; 7 8')
L = A.dot(B + C)
R = A.dot(B) + A.dot(C)
print(L)
print(R)

[[35 42]
 [77 94]]
[[35 42]
 [77 94]]
```

```
[6]: A = np.matrix('1 2; 3 4')
B = np.matrix('5 6; 7 8')
L = A.dot(B)
R = B.dot(A)
print(L)
print(R)

[[19 22]
 [43 58]]
[[23 34]
 [31 46]]
```

```
[7]: A = np.matrix('1 2; 3 4')
E = np.matrix('1 0; 0 1')
L = E.dot(A)
R = A.dot(E)
print(L)
print(R)
print(A)

[[1 2]
 [3 4]]
[[1 2]
 [3 4]]
[[1 2]
 [3 4]]
```

```
[8]: A = np.matrix('1 2; 3 4')
Z = np.matrix('0 0; 0 0')
L = Z.dot(A)
R = A.dot(Z)
print(L)
print(R)
print(Z)

[[0 0]
 [0 0]]
[[0 0]
 [0 0]]
[[0 0]
 [0 0]]
```

Рисунок 26. Пример 22

```

[1]: import numpy as np
A = np.matrix('-4 -1 2; 10 4 -1; 8 3 1')
print(A)

[[-4 -1 2]
 [10  4 -1]
 [ 8  3 1]]

[3]: print(np.linalg.det(A))

-14.000000000000009

[4]: A = np.matrix('-4 -1 2; 10 4 -1; 8 3 1')
print(A)
print(A.T)
det_A = round(np.linalg.det(A), 3)
det_A_t = round(np.linalg.det(A.T), 3)
print(det_A)
print(det_A_t)

[[-4 -1 2]
 [10  4 -1]
 [ 8  3 1]]
[[-4 10 8]
 [-1  4 3]
 [ 2 -1 1]]
-14.0
-14.0

[5]: A = np.matrix('-4 -1 2; 0 0 0; 8 3 1')
print(A)
print(np.linalg.det(A))

[[-4 -1 2]
 [ 0  0 0]
 [ 8  3 1]]
0.0

[6]: A = np.matrix('-4 -1 2; 10 4 -1; 8 3 1')
B = np.matrix('10 4 -1; -4 -1 2; 8 3 1')
print(A)
print(B)
print(round(np.linalg.det(A), 3))
print(round(np.linalg.det(B), 3))

[[-4 -1 2]
 [10  4 -1]
 [ 8  3 1]]
[[10  4 -1]
 [-4 -1 2]
 [ 8  3 1]]
-14.0
14.0

[7]: A = np.matrix('-4 -1 2; -4 -1 2; 8 3 1')
print(A)
print(np.linalg.det(A))

[[-4 -1 2]
 [-4 -1 2]
 [ 8  3 1]]
0.0

```

Рисунок 27. Пример 23 (1 часть)

```
[8]: A = np.matrix(' -4 -1 2; 10 4 -1; 8 3 1')
print(A)
k = 2
B = A.copy()
B[2, :] = k * B[2, :]
print(B)
det_A = round(np.linalg.det(A), 3)
det_B = round(np.linalg.det(B), 3)
print(det_A * k)
print(det_B)

[[-4 -1 2]
 [10 4 -1]
 [ 8 3 1]]
[[-4 -1 2]
 [10 4 -1]
 [16 6 2]]
-28.0
-28.0
```

```
[9]: A = np.matrix(' -4 -1 2; -4 -1 2; 8 3 1')
B = np.matrix(' -4 -1 2; 8 3 2; 8 3 1')
C = A.copy()
C[1, :] += B[1, :]
print(C)
print(A)
print(B)
print(round(np.linalg.det(C), 3))
print(round(np.linalg.det(A), 3) + (round(np.linalg.det(B), 3)))

[[-4 -1 2]
 [ 4 2 4]
 [ 8 3 1]]
[[-4 -1 2]
 [-4 -1 2]
 [ 8 3 1]]
[[-4 -1 2]
 [ 8 3 2]
 [ 8 3 1]]
4.0
4.0
```

```
[10]: A = np.matrix(' -4 -1 2; 10 4 -1; 8 3 1')
k = 2
B = A.copy()
B[1, :] = B[1, :] + k * B[0, :]
print(A)
print(B)
print(round(np.linalg.det(A), 3))
print(round(np.linalg.det(B), 3))

[[-4 -1 2]
 [10 4 -1]
 [ 8 3 1]]
[[-4 -1 2]
 [ 2 2 3]
 [ 8 3 1]]
-14.0
-14.0
```

```
[11]: A = np.matrix(' -4 -1 2; 10 4 -1; 8 3 1')
print(A)
k = 2
A[1, :] = A[0, :] + k * A[2, :]
print(round(np.linalg.det(A), 3))

[[-4 -1 2]
 [10 4 -1]
 [ 8 3 1]]
0.0
```

Рисунок 28. Пример 23 (2 часть)

```
[12]: A = np.matrix(' -4 -1 2; 10 4 -1; 8 3 1')
print(A)
k = 2
A[1, :] = k * A[0, :]
print(A)
print(round(np.linalg.det(A), 3))

[[-4 -1 2]
 [10 4 -1]
 [ 8 3 1]]
[[-4 -1 2]
 [-8 -2 4]
 [ 8 3 1]]
0.0
```

Рисунок 28. Пример 23 (3 часть)

```
[2]: import numpy as np
A = np.matrix('1 -3; 2 5')
A_inv = np.linalg.inv(A)
print(A_inv)

[[ 0.45454545  0.27272727]
 [-0.18181818  0.09090909]]

[3]: A = np.matrix('1 -3; 2 5')
A_inv = np.linalg.inv(A)
A_inv_inv = np.linalg.inv(A_inv)
print(A)
print(A_inv_inv)

[[ 1 -3]
 [ 2  5]]
[[ 1. -3.]
 [ 2.  5.]]

[4]: A = np.matrix('1 -3; 2 5')
L = np.linalg.inv(A.T)
R = (np.linalg.inv(A)).T
print(L)
print(R)

[[ 0.45454545 -0.18181818]
 [ 0.27272727  0.09090909]]
[[ 0.45454545 -0.18181818]
 [ 0.27272727  0.09090909]]

[5]: A = np.matrix('1 -3; 2 5')
B = np.matrix('7 6; 1 8')
L = np.linalg.inv(A.dot(B))
R = np.linalg.inv(B).dot(np.linalg.inv(A))
print(L)
print(R)

[[ 0.09454545  0.03272727]
 [-0.03454545  0.00727273]]
[[ 0.09454545  0.03272727]
 [-0.03454545  0.00727273]]
```

Рисунок 29. Пример 24

```
[2]: import numpy as np
      m_eye = np.eye(4)
      print(m_eye)

      [[1.  0.  0.  0.]
       [0.  1.  0.  0.]
       [0.  0.  1.  0.]
       [0.  0.  0.  1.]]

[3]: rank = np.linalg.matrix_rank(m_eye)
      print(rank)

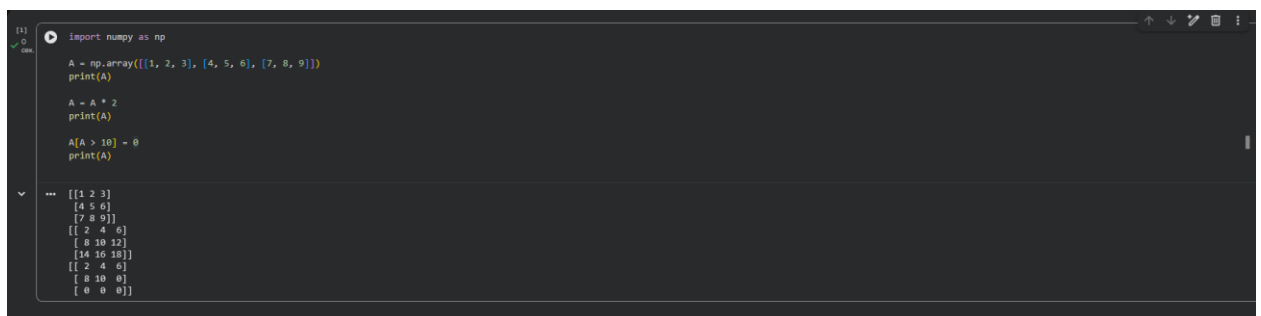
      4

[4]: m_eye[3][3] = 0
      print(m_eye)
      rank = np.linalg.matrix_rank(m_eye)
      print(rank)

      [[1.  0.  0.  0.]
       [0.  1.  0.  0.]
       [0.  0.  1.  0.]
       [0.  0.  0.  0.]]
      3
```

Рисунок 30. Пример 25

5. Выполнили задания практической работы.



```
[1]: import numpy as np

      A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
      print(A)

      A = A * 2
      print(A)

      A[A > 10] = 0
      print(A)

      ... [[1 2 3]
           [4 5 6]
           [7 8 9]]
           [[2 4 6]
           [8 10 12]
           [14 16 18]]
           [[2 4 6]
           [8 10 0]
           [0 0 0]]
```

Рисунок 31. Задание 1

```
import numpy as np
arr = np.random.randint(1, 101, 20)
print("Исх массив:")
print(arr)

a = arr % 5 == 0
print("\nЭл, дел на 5:")
print(arr[a])

arr[a] = -1
print("\nОбнов.массив (эл, дел на 5, зам на -1):")
print(arr)
```

Исх массив:
[79 53 51 4 37 37 68 72 82 60 12 21 1 51 4 82 73 77 65 34]

Эл, дел на 5:
[60 65]

Обнов.массив (эл, дел на 5, зам на -1):
[79 53 51 4 37 37 68 72 82 -1 12 21 1 51 4 82 73 77 -1 34]

Рисунок 32. Задание 2

```
import numpy as np
arr1 = np.random.randint(0, 51, 5)
arr2 = np.random.randint(0, 51, 5)
print(arr1)
print(arr2)

a = np.vstack([arr1, arr2])
print("\nДвумерный массив:")
print(a)

b, c = a[0, :], a[1, :]
print("\nРазделенные массивы:")
print("Первый:", b)
print("Второй:", c)
```

[23 48 28 5 2]
[17 11 7 3 8]

Двумерный массив:
[[23 48 28 5 2]
 [17 11 7 3 8]]

Разделенные массивы:
Первый: [23 48 28 5 2]
Второй: [17 11 7 3 8]

Рисунок 33. Задание 3

```
import numpy as np
arr = np.linspace(-10, 10, 50)
print("Массив из 50 чисел от -10 до 10:")
print(arr)

total_sum = np.sum(arr)
print(f"\nСумма всех элементов: {total_sum}")
plus_sum = np.sum(arr[arr > 0])
print(f"Сумма положительных элементов: {plus_sum:.4f}")
minus_sum = np.sum(arr[arr < 0])
print(f"Сумма отрицательных элементов: {minus_sum:.4f}")
```

Массив из 50 чисел от -10 до 10:
[-10. -9.59183673 -9.18367347 -8.7755102 -8.36734694
 -7.95918367 -7.55102041 -7.14285714 -6.73469388 -6.32653061
 -5.91836735 -5.51020408 -5.10204082 -4.69387755 -4.28571429
 -3.87755102 -3.46938776 -3.06122449 -2.65306122 -2.24489796
 -1.83673469 -1.42857143 -1.02040816 -0.6122449 -0.20408163
 0.20408163 0.6122449 1.02040816 1.42857143 1.83673469
 2.24489796 2.65306122 3.06122449 3.46938776 3.87755102
 4.28571429 4.69387755 5.10204082 5.51020408 5.91836735
 6.32653061 6.73469388 7.14285714 7.55102041 7.95918367
 8.36734694 8.7755102 9.18367347 9.59183673 10.]

Сумма всех элементов: 7.105427357601002e-15
Сумма положительных элементов: 127.5510
Сумма отрицательных элементов: -127.5510

Рисунок 34. Задание 4

```
import numpy as np
edmat = np.eye(4)
print("Единичная матрица 4x4:")
print(edmat)

d = np.diag([5, 10, 15, 20])
print("\nДиагональная матрица:")
print(d)

sum_edmat = np.sum(edmat)
sum_d = np.sum(d)

print(f"\nСумма элементов единичной матрицы: {sum_edmat}")
print(f"Сумма элементов диагональной матрицы: {sum_d}")
print(f"Разница: {sum_d - sum_edmat}")

*** Единичная матрица 4x4:
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]

Диагональная матрица:
[[ 5  0  0  0]
 [ 0 10  0  0]
 [ 0  0 15  0]
 [ 0  0  0 20]]

Сумма элементов единичной матрицы: 4.0
Сумма элементов диагональной матрицы: 50
Разница: 46.0
```

Рисунок 35. Задание 5


```

import numpy as np
A = np.random.randint(1, 21, (3, 3))
B = np.random.randint(1, 21, (3, 3))

print("Матрица A:")
print(A)
print("\nМатрица B:")
print(B)

smatr = A + B
print("\nСумма A + B:")
print(smatr)

raznat = A - B
print("\nРазность A - B:")
print(raznat)

poelement = A * B
print("\nПэлементное произведение:")
print(poelement)

```

```

*** Матрица A:
[[ 2 12 11]
 [17  8  8]
 [ 9  2  3]]

Матрица B:
[[ 5 13 10]
 [20  1 16]
 [11 11 18]]

Сумма A + B:
[[ 7 25 21]
 [37  9 24]
 [20 13 21]]

Разность A - B:
[[ -3 -1  1]
 [ -3  7 -8]
 [ -2 -9 -15]]

Пэлементное произведение:
[[ 10 156 110]
 [340  8 128]
 [ 99 22 54]]

```

Рисунок 36. Задание 6

```
import numpy as np
A = np.random.randint(1, 11, (2, 3))
print("Матрица A (2x3):")
print(A)

B = np.random.randint(1, 11, (3, 2))
print("\nМатрица B (3x2):")
print(B)

result = np.dot(A, B)
print("\nРезультат умножения A * B (2x2):")
print(result)
```

```
Матрица A (2x3):
[[5 7 8]
 [5 5 2]]
```

```
Матрица B (3x2):
[[3 2]
 [5 9]
 [6 4]]
```

```
Результат умножения A * B (2x2):
[[ 98 105]
 [ 52  63]]
```

Рисунок 37. Задание 7

```

import numpy as np
A = np.random.rand(3, 3)
print("Матрица A:")
print(A)

opr = np.linalg.det(A)
print(f"\nОпределитель матрицы: {opr:.4f}")

if opr == 0:
    print("\nМатрица вырождена")
else:
    opr = np.linalg.inv(A)
    print("\nОбратная матрица:\n", opr)

```

*** Матрица A:

```

[[0.22483293 0.99120794 0.81278172]
 [0.44662988 0.65878995 0.3736131 ]
 [0.49621505 0.584385   0.37897482]]

```

Определитель матрицы: -0.0305

Обратная матрица:

```

[[ -1.02633198  -3.25398361   5.40910545]
 [ -0.52841509  10.42053765  -9.13982424]
 [  2.15866379 -11.80798829   9.64997264]]

```

Рисунок 38. Задание 8

```

import numpy as np
A = np.random.randint(1, 51, (4, 4))
print("Матрица 4x4:")
print(A)

transposed = np.transpose(A)
print("\nТранспонированная матрица:")
print(transposed)

trace = np.trace(A)
print(f"\nСлед матрицы: {trace}")

```

Матрица 4x4:

```

[[21  9 43 27]
 [ 4 25 33  2]
 [19 25 14 39]
 [12 29  3 39]]

```

Транспонированная матрица:

```

[[21  4 19 12]
 [ 9 25 25 29]
 [43 33 14  3]
 [27  2 39 39]]

```

След матрицы: 99

Рисунок 39. Задание 9

```
import numpy as np

A = np.array([[2, 3, -1], [4, -1, 2], [-3, 5, 4]])
B = np.array([[5], [6], [-2]])

print("Матрица коэффициентов A:")
print(A)
print("\nВектор правой части B:")
print(B)

res = np.linalg.solve(A, B)

print("\nРешение системы:")
print(f"x = {res[0][0]:.4f}")
print(f"y = {res[1][0]:.4f}")
print(f"z = {res[2][0]:.4f}")

*** Матрица коэффициентов A:
[[ 2  3 -1]
 [ 4 -1  2]
 [-3  5  4]]

Вектор правой части B:
[[ 5]
 [ 6]
 [-2]]

Решение системы:
x = 1.6396
y = 0.5766
z = 0.0090
```

Рисунок 40. Задание 10

Индивидуальное задание:

Распределение инвестиций. Компания выделила 1 000 000 рублей на три проекта. Первый проект получает в два раза больше, чем второй, а третий получает на 200 000 рублей больше, чем второй. Сколько денег получит каждый проект?

Решение:

```
import numpy as np

A = np.array([[1, 1, 1], [1, -2, 0], [0, -1, 1]], dtype=float)
B = np.array([1000000, 0, 200000], dtype=float)

A_inv = np.linalg.inv(A)
res_matrix = A_inv.dot(B)
print(res_matrix)

det_A = np.linalg.det(A)
A1 = A.copy(); A1[:, 0] = B
A2 = A.copy(); A2[:, 1] = B
A3 = A.copy(); A3[:, 2] = B

x_kramer = np.linalg.det(A1) / det_A
y_kramer = np.linalg.det(A2) / det_A
z_kramer = np.linalg.det(A3) / det_A
print(np.array([x_kramer, y_kramer, z_kramer]))

res_solve = np.linalg.solve(A, B)
print(res_solve)

[400000. 200000. 400000.]
[400000. 200000. 400000.]
[400000. 200000. 400000.]
```

Рисунок 41. Выполнение индивидуального задания

Контрольные вопросы:

1. NumPy (Numerical Python) – это фундаментальная библиотека для научных вычислений в Python. Она предоставляет:

- Многомерные массивы (ndarray) высокой производительности.
- Эффективные векторные и матричные операции.
- Широкий набор математических функций, работающих с массивами.
- Средства для линейной алгебры, преобразования Фурье, генерации случайных чисел и т.д.
- Основу для многих других библиотек (SciPy, pandas, scikit-learn, TensorFlow и др.).

2. Основная структура данных в NumPy – это многомерный массив, называемый ndarray. Это объект, который представляет собой таблицу данных (например, вектор или матрицу) и позволяет выполнять операции с ними с

высокой эффективностью.

3. Доступ к частям многомерного массива осуществляется с помощью индексации и срезов.

4. NumPy предоставляет множество функций для расчёта статистических характеристик. Они могут применяться ко всему массиву или вдоль указанной оси.

5. Выборка – получение подмножества элементов. Способы: срезы, передача списков/массивов индексов, булева индексация.

6. Основные виды матриц и векторов:

- Вектор одномерный массив (столбцовый или строчный). $v = \text{np.array}([1,2,3])$

- Матрица – двумерный массив. $M = \text{np.array}([[1,2],[3,4]])$

- Квадратная матрица – число строк = число столбцов.

- Единичная матрица – единицы на главной диагонали. $I = \text{np.eye}(3)$

- Диагональная матрица – ненулевые элементы только на диагонали.

$D = \text{np.diag}([1,2,3])$

- Симметричная матрица – $A = A^T$.

7. Транспонирование – операция, при которой строки матрицы становятся столбцами, а столбцы – строками.

8. Свойства:

– Дважды транспонированная матрица равна исходной матрице

– Транспонирование суммы матриц равно сумме транспонированных матриц

– Транспонирование произведения матриц равно произведению транспонированных матриц расставленных в обратном порядке

– Транспонирование произведения матрицы на число равно произведению этого числа на транспонированную матрицу

– Определители исходной и транспонированной матрицы совпадают

9. Средства NumPy для транспонирования: атрибут `.T` — наиболее удобный; функция `np.transpose(a, axes=None)` — позволяет переставлять оси многомерного массива.

10. Основные действия над матрицами:

- Сложение и вычитание
- Умножение на число
- Умножение матриц (скалярное, матричное, поэлементное)
- Транспонирование
- Вычисление определителя (для квадратных)
- Нахождение обратной матрицы

11. При умножении матрицы на число, все элементы матрицы умножаются на это число.

12. Свойства:

- Произведение единицы и любой заданной матрицы равно заданной матрице.
- Произведение нуля и любой матрицы равно нулевой матрице, размерность которой равна исходной матрицы.
- Произведение матрицы на сумму чисел равно сумме произведений матрицы на каждое из этих чисел.
- Произведение матрицы на произведение двух чисел равно произведению второго числа и заданной матрицы, умноженному на первое число.
- Произведение суммы матриц на число равно сумме произведений этих матриц на заданное число.

13. Складывать (вычитать) можно только матрицы одинаковой размерности – то есть матрицы, у которых совпадает количество столбцов и строк.

14) Свойства:

- Коммутативность сложения. От перестановки матриц их сумма не

изменяется.

- Ассоциативность сложения. Результат сложения трех и более матриц не зависит от порядка, в котором эта операция будет выполняться.

- Для любой матрицы существует противоположная ей, такая, что их сумма является нулевой матрицей.

15) Средства NumPy для сложения и вычитания: операторы `+` и `-`; функции `np.add()`, `np.subtract()`; `np.sum()` для суммирования всех элементов или по оси.

16) Умножение матриц это уже более сложная операция, по сравнению с рассмотренными выше. Умножать можно только матрицы, отвечающие следующему требованию: количество столбцов первой матрицы должно быть равно числу строк второй матрицы.

17) Свойства:

- Ассоциативность умножения. Результат умножения матриц не зависит от порядка, в котором будет выполняться эта операция.

- Дистрибутивность умножения. Произведение матрицы на сумму матриц равно сумме произведений матриц.

- Умножение матриц в общем виде не коммутативно. Это означает, что для матриц не выполняется правило независимости произведения от перестановки множителей.

- Произведение заданной матрицы на единичную равно исходной матрице.

- Произведение заданной матрицы на нулевую матрицу равно нулевой матрице.

18) Средства NumPy для умножения матриц: `@` — оператор (начиная с Python 3.5); `np.dot()` — также работает для скалярного произведения векторов.

19) Определитель матрицы размера (n-го порядка) является одной из ее численных характеристик. Определитель матрицы A обозначается как $|A|$ или $\det(A)$, его также называют детерминантом.

Свойства:

- Определитель матрицы остается неизменным при ее транспонировании.
- Если у матрицы есть строка или столбец, состоящие из нулей, то определитель такой матрицы равен нулю.
- При перестановке строк матрицы знак ее определителя меняется на противоположный.
- Если у матрицы есть две одинаковые строки, то ее определитель равен нулю.
- Если все элементы строки или столбца матрицы умножить на какое-то число, то и определитель будет умножен на это число.
- Если все элементы строки или столбца можно представить как сумму двух слагаемых, то определитель такой матрицы равен сумме определителей двух соответствующих матриц.
- Если к элементам одной строки прибавить элементы другой строки, умноженные на одно и то же число, то определитель матрицы не изменится.
- Если строка или столбец матрицы является линейной комбинацией других строк (столбцов), то определитель такой матрицы равен нулю.
- Если матрица содержит пропорциональные строки, то ее определитель равен нулю.

20) Средства NumPy для нахождения определителя: функция `np.linalg.det(A)`. Возвращает значение типа `float` (или комплексное).

21) Обратной матрицей A^{-1} матрицы A называют матрицу, удовлетворяющую следующему равенству: $A * A^{-1} = A^{-1} \times A = E$, где E - это единичная матрица.

Алгоритм нахождения:

- Вычислить определитель.
- Найти матрицу алгебраических дополнений.
- Транспонировать её — получить присоединённую матрицу.

– Разделить присоединённую матрицу на определитель.

22) Свойства:

– Обратная матрица обратной матрицы есть исходная матрица.

– Обратная матрица транспонированной матрицы равна транспонированной матрице от обратной матрицы.

– Обратная матрица произведения матриц равна произведению обратных матриц.

23) Функция `np.linalg.inv(A)` возвращает обратную матрицу, если она существует.

24) Каждая переменная выражается через определитель матрицы, где i -й столбец заменяется столбцом свободных членов. Формально это записывается как $x_i = \det(A_i) / \det(A)$, где A_i — матрица, полученная из A заменой i -го столбца на вектор b .

```
det_A = np.linalg.det(A)
if det_A == 0:
    print("Матрица вырождена, метод Крамера неприменим.")
else:
    x_cramer = []
    for i in range(3):
        Ai = A.copy()
        Ai[:, i] = B
        det_Ai = np.linalg.det(Ai)
        x_cramer.append(det_Ai / det_A)
    x_cramer = np.array(x_cramer)
    print("Метод Крамера:", x_cramer)
```

25) Матричный метод – это метод решения систем линейных уравнений, основанный на использовании обратной матрицы.

```
if det_A == 0:
    print("Матрица вырождена, обратная матрица не существует.")
else:
```

```
A_inv = np.linalg.inv(A)
x_inv = A_inv @ B
print("Матричный метод:", x_inv)
```

Вывод: Были освоены ключевые возможности и методы библиотеки NumPy для программирования на Python.